

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – CODING CHALLENGE QUESTIONS

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO1 – Precision (BL4)	Assessment:	Coding challenge questions
Weightage:	40%	Total Marks:	100
CO1 Statement:	Analyse the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking		
Student Name:	R Hardhik	Reg. No.:	192312417
Section / Batch:	2023	Date:	29 - 07 - 2026

Instructions to Students

- Analyze the given scenario and identify the problem requirements before developing the solution.
- Design the algorithm or flowchart and select the appropriate 8085 instructions, registers, and addressing modes required for implementation.
- Develop and execute the 8085 Assembly Language Program (ALP) using a simulator or trainer kit to solve the given problem.
- Verify and validate the output by testing the program with the given input data and ensuring the expected results are obtained.
- Interpret the program execution by explaining the logic used, the role of registers, memory locations, flags, and the final output generated.

QUESTIONS

1. An intelligent traffic controller records the number of vehicles passing through four junctions every hour. The controller must determine which junction has the highest traffic and calculate the total number of vehicles recorded. Write an 8085 ALP that
 - Finds the busiest junction.
 - Calculates the total traffic count.
 - Compares the total with a predefined congestion threshold.
 - Stores a congestion indicator if the threshold is exceeded.
2. An embedded battery management system stores voltage readings collected during charging. Any reading below the safe voltage level indicates a faulty battery cell. Write an 8085 ALP to
 - Count the number of unsafe voltage readings.
 - Find the minimum voltage recorded.
 - Store both results in memory.
3. A processor executes a sequence of instruction types represented by numeric codes stored in memory:
 - 01H – Data Transfer
 - 02H – Arithmetic
 - 03H – Logical
 - 04H – Branch

The system must determine how many instructions belong to each category. Develop an 8085 ALP that scans the instruction sequence and counts the occurrence of each instruction category.

Code of Conduct

I R. Hardhik (192312417) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Q. No	Problem Understanding (20)	Algorithm (25)	Code Implementation (20)	Competitive Performance (20)	Test case handling (15)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 25	/ 20	/ 20	/ 15	/ 100

Course Coordinator

Faculty Incharge

Q1:

GNUSim8085 - 8085 Microprocessor Simulator

File Reset Assembler Debug Help

Registers: A 01, BC 46 71, DE 00 00, HL 08 05, PSW 00 00, PC 42 3D, SP FF FF, Int-Reg 00 00. Flag: S 0, Z 0, AC 0, P 1, C 0.

Decimal - Hex Conversion: Decimal 0, Hex 0. To Hex, To Dec.

I/O Ports: 0, Update Port Value.

Memory: 0, Update Memory.

Load me at: 1 LXI H,2050, 2 MOV A,M, 3 MOV B,A, 4 MOV C,A, 5 INX H, 6 MOV A,C, 7 ADD M, 8 MOV C,A, 9 MOV A,B, 10 JNC SKIP1, 11 JNC B,R, 12 JNC SKIP1, 13 JNC B,R, 14 JNC SKIP1, 15 JNC B,R, 16 JNC SKIP1, 17 JNC B,R, 18 JNC SKIP1, 19 JNC B,R, 20 JNC SKIP1, 21 JNC B,R, 22 JNC SKIP1, 23 JNC B,R, 24 JNC SKIP1, 25 JNC B,R, 26 JNC SKIP1, 27 JNC B,R, 28 JNC SKIP1, 29 JNC B,R, 30 JNC SKIP1, 31 JNC B,R, 32 JNC SKIP1, 33 JNC B,R, 34 JNC SKIP1, 35 JNC SKIP1, 36 JNC SKIP1, 37 JNC SKIP1, 38 JNC SKIP1, 39 JNC SKIP1, 40 JNC SKIP1, 41 JNC SKIP1, 42 JNC SKIP1, 43 JNC SKIP1, 44 JNC SKIP1, 45 JNC SKIP1, 46 JNC SKIP1, 47 JNC SKIP1, 48 JNC SKIP1, 49 JNC SKIP1, 50 JNC SKIP1, 51 JNC SKIP1, 52 JNC SKIP1, 53 JNC SKIP1, 54 JNC SKIP1, 55 JNC SKIP1, 56 JNC SKIP1.

Memory dump: Start 2050, Address (Hex) Address Data, 0802 2050 20, 0803 2051 35, 0804 2052 18, 0805 2053 40, 0806 2054 40, 0807 2055 113, 0808 2056 1, 0809 2057 70, 080A 2058 0, 080B 2059 0, 080C 2060 0, 080D 2061 0, 080E 2062 0, 080F 2063 0, 0810 2064 0, 0811 2065 0, 0812 2066 0, 0813 2067 0, 0814 2068 0, 0815 2069 0, 0816 2070 0, 0817 2071 0.

Line No Assembler Message: 0 Program assembled successfully

Simulator: Idle

Q2:

GNUSim8085 - 8085 Microprocessor Simulator

File Reset Assembler Debug Help

Registers: A 1E, BC 1E 02, DE 00 00, HL 08 07, PSW 00 00, PC 42 26, SP FF FF, Int-Reg 00 00. Flag: S 0, Z 1, AC 0, P 1, C 0.

Decimal - Hex Conversion: Decimal 0, Hex 0. To Hex, To Dec.

I/O Ports: 0, Update Port Value.

Memory: 0, Update Memory.

Load me at: 1 LXI H,2050, 2 MOV A,M, 3 MOV B,A, 4 MOV C,08, 5 MOV D,05, 6 JNC B,R, 7 JNC B,R, 8 JNC B,R, 9 JNC B,R, 10 JNC B,R, 11 JNC B,R, 12 JNC B,R, 13 JNC B,R, 14 JNC B,R, 15 JNC B,R, 16 JNC B,R, 17 JNC B,R, 18 JNC B,R, 19 JNC B,R, 20 JNC B,R, 21 JNC B,R, 22 JNC B,R, 23 JNC B,R, 24 JNC B,R, 25 JNC B,R, 26 JNC B,R, 27 JNC B,R, 28 JNC B,R, 29 JNC B,R, 30 JNC B,R, 31 JNC B,R, 32 JNC B,R, 33 JNC B,R, 34 JNC B,R, 35 JNC B,R, 36 JNC B,R, 37 JNC B,R, 38 JNC B,R, 39 JNC B,R, 40 JNC B,R, 41 JNC B,R, 42 JNC B,R, 43 JNC B,R, 44 JNC B,R, 45 JNC B,R, 46 JNC B,R, 47 JNC B,R, 48 JNC B,R, 49 JNC B,R, 50 JNC B,R, 51 JNC B,R, 52 JNC B,R, 53 JNC B,R, 54 JNC B,R, 55 JNC B,R, 56 JNC B,R.

Memory dump: Start 2050, Address (Hex) Address Data, 0802 2050 50, 0803 2051 42, 0804 2052 38, 0805 2053 55, 0806 2054 30, 0807 2055 40, 0808 2056 2, 0809 2057 30, 080A 2058 0, 080B 2059 0, 080C 2060 0, 080D 2061 0, 080E 2062 0, 080F 2063 0, 0810 2064 0, 0811 2065 0, 0812 2066 0, 0813 2067 0, 0814 2068 0, 0815 2069 0, 0816 2070 0, 0817 2071 0.

Line No Assembler Message: 0 Program assembled successfully

Simulator: Idle

Q3:

GNUSim8085 - 8085 Microprocessor Simulator

File Reset Assembler Debug Help

Registers

A	03	S	0
BC	03 02	Z	1
DE	02 03	AC	0
HL	08 0C	P	1
PSW	00 00	C	0
PC	42 4E		
SP	FF FF		

Int-Reg 00

Decimal - Hex Conversion

Decimal	Hex
0	0

To Hex To Dec

I/O Ports

0	-	+	00
---	---	---	----

Update Port Value

Memory

0	-	+	00
---	---	---	----

Update Memory

Load me at:

1 LXI H,2050

2

3 MVI B,00H

4 MVI C,00H

5 MVI D,00H

6 MVI E,00H

7

8 MVI A,00H

9 STA 2050H

10

11 LOOP: MOV A,H

12

13 CPI B,H

14 JNZ CH2

15 INR B

16 JMP NEXT

17

18 CH2: CPI 02H

19 JNZ CH3

20 INR C

21 JMP NEXT

22

23 CH3: CPI 03H

24 JNZ CH4

25 INR D

26 JMP NEXT

27

28 CH4: CPI 04H

29 JNZ NEXT

30 INR E

31

32 NEXT: INX H

33 LDA 2050H

34 DCR A

35 STA 2050H

36 JNC LOOP

37

38 MOV A,B

39 STA 2050H

40

41 MOV A,C

42 STA 2050H

43

44 MOV A,D

45 STA 2050H

46

47 MOV A,E

48 STA 2050H

49

50 HLT

Data Stack KeyPad Memory I/O Ports

Start 2050 OK

Address (Hex) Address Data

0802	2050	1
0803	2051	2
0804	2052	1
0805	2053	3
0806	2054	4
0807	2055	2
0808	2056	3
0809	2057	4
080A	2058	4
080B	2059	1
080C	2060	3
080D	2061	2
080E	2062	2
080F	2063	3
0810	2064	0
0811	2065	0
0812	2066	0
0813	2067	0
0814	2068	0
0815	2069	0
0816	2070	0
0817	2071	0

Line No Assembler Message

0 Program assembled successfully

Simulator: Idle